

Tor Website Fingerprinting

Önálló laboratórium beszámoló

Pálos Ferenc

2026. május 14.

Összefoglaló

A beszámoló a Tor hálózaton végzett weboldal ujjlenyomat-készítés (Website Fingerprinting, WF) feladatát mutatja be, két eltérő modelles család összehasonlításával: klasszikus gépi tanulás (Random Forest) és mélytanulás (Triplet MLP + KNN). A vizsgálat a zárt világú (baseline, obfs4), a zero-shot és a nyílt világú (open-world) forgatókönyvekben méri a teljesítményt. A végső értékelés a futtatott Python pipeline által számolt metrikákon és a generált ábrákon alapul. A Random Forest erős baseline és obfs4 teljesítményt ad, míg a Triplet MLP a zero-shot beállításokban kedvezőbb általánosítást mutat. A dokumentum részletesen bemutatja az adatgyűjtési és reprodukálhatósági lépéseket is, hogy a pipeline végponttól végpontig újrafuttatható legyen.

1 Célkitűzések és hozzájárulások

A projekt célja egy reprodukálható WF pipeline kialakítása és két modelles család objektív összehasonlítása a Tor forgalomban jelentkező domain shift és ismeretlen forgalom kezelésére. Ennek keretében:

- a PCAP állományoktól a kiértékelési metrikákig egy végig automatizált feldolgozási láncot raktam össze,
- aggregált folyamjellemzőket definiáltam és egységes 21-dimenziós feature teret alkalmaztam,
- *shared* és *optimized* top-10 feature track-eket vezettem be,
- egységes ábragenerálást biztosítottam a kiértékelési JSON-okból,
- az open-world „Other” osztály kezelését és a zero-shot beállításokat külön mérési szálként kezeltem.

2 Adatok és forgatókönyvek

A mérés három adatdoménen történik: baseline Tor forgalom, obfs4 Tor forgalom, valamint az open-world „Other” kategória. A fő forgatókönyvek:

- **Baseline (closed-world):** standard Tor böngészés.

- **Obfs4 (closed-world):** obfs4 pluggable transport által elfedett forgalom.
- **Zero-shot:** baseline-on tanított modellek tesztelése obfs4 forgalmon.
- **Open-world:** „Other” kategória hozzáadása az ismeretlen oldalak modellezésére.

A zárt világú osztályokat rétegzett 80%/20% tanító/teszt felosztással értékelem. Az open-world „Other” osztályt ezzel szemben 50%–50% arányban bontom tanító/teszt részre, mert itt a cél nem az adott „Other” webhelyek pontos felismerése, hanem az ismeretlen forgalomra való általánosítás. A nagyobb „Other” teszt rész valóságosabb „ismeretlen” mintaállományt ad, csökkenti a recall becslés zaját, és akkor is értelmezhető méretű tesztet hagy, ha az „Other” mintaszám kisebb.

3 Adatgyűjtés és előfeldolgozás

Az adatgyűjtés automatizált headless böngészéssel és `tcpdump`-pal történik, aktív Tor SOCKS proxy és ControlPort mellett. A SOCKS proxy a Tor hálózatra irányítja a böngésző forgalmát, míg a ControlPort a Tor példány vezérlését teszi lehetővé (pl. új áramkör kérése, állapotlekérdezés). A zárt világú baseline/obfs4 forgalmat a `scripts/collection/collector.py` (futtató: `run_collector.sh`) gyűjti, míg az open-world „Other” adatokat a `server_collect_other.py` szkript (futtató: `run_server_collect_other.sh`) állítja elő és a `tor_dataset/other_tor/` könyvtárba menti. A gyűjtés környezetét opcionálisan környezeti változók (pl. `TOR_WF_PCAP_DIR`, `TOR_WF_INTERFACE`, `TOR_WF_GUARD_IP`, `TOR_WF_CAPTURE_DURATION`, `TOR_WF_WARMUP_DURATION`) szabályozzák. Itt a `TOR_WF_INTERFACE` adja meg a `tcpdump` által figyelt hálózati interfészt (pl. `eth0` vagy `wlan0`), a `TOR_WF_WARMUP_DURATION` pedig a tényleges mérés előtti bemelegítési időt másodpercben, amely a Tor/circuit stabilizálását szolgálja.

4 Feldolgozási pipeline és jellemzők

A pipeline három lépésből áll. **(1)** A PCAP fájlokból kiolvasom az egyes csomagokat, és csomagonkénti idősorokat készítek (mikor jött a csomag, mekkora volt, milyen irányban haladt). **(2)** Ezekből az idősorokból egy-egy kapcsolatot összefoglaló, aggregált jellemzőket számolok. Ilyenek például az összes be- és kimenő csomagszám, a be- és kimenő bájtok összege, a csomagok közti érkezési idők statisztikái (átlag, szórás, kvantilisok), a kapcsolat időtartama, illetve az irányváltások száma. **(3)** A jellemzőkön tanítom és értékelem a modelleket. Az alap 21 jellemzőből minden futásban top-10 készletet választok, így a modellek összehasonlítása egységes dimenziószámon történik. A jellemzők főbb csoportjait a 1 táblázat foglalja össze.

1. táblázat. Aggregált folyamjellelmezők csoportjai és példák.

Csoport	Jellemzők (példák)
Csomagszámok	<code>total_pkts</code> , <code>in_pkts</code> , <code>out_pkts</code>
Bájtstatisztikák	<code>total_bytes</code> , <code>in_bytes</code> , <code>out_bytes</code> , <code>in/out ratio</code>
Időköz-statisztikák	<code>iat_mean</code> , <code>iat_std</code> , <code>iat_median</code> , <code>iat_p90</code>
Méretstatisztikák	<code>size_mean</code> , <code>size_std</code> , <code>size_median</code> , <code>size_p90</code>
Időtartam és dinamika	<code>duration</code> , <code>direction_changes</code>

5 Modellek és kiértékelés

Random Forest. 300 döntési fából álló modell, stabilizált top-10 jellemzőkiválasztással és három véletlen maggal (42, 52, 62). A három mag segítségével a tanító/teszt felosztást és a modell tanítását háromszor ismétlem, majd átlagot és szórást számolok.

Triplet MLP + KNN. A Triplet MLP 32 dimenziós beágyazást tanul. A Triplet Margin Loss arra kényszeríti a modellt, hogy az azonos weboldalhoz tartozó minták beágyazása közel legyen, míg a különböző weboldalaké távol. A beágyazásokon $k = 5$ KNN osztályozást végzek.

Feature track-ek. *shared* track-ben egy közös top-10 készletet választok ki a mutual information alapján. *optimized* track-ben forgatókönyvenként külön top-10 készlet készül (RF: több futásból stabilizált fontossági rangsor, DL: mutual information).

Mélytanulási beállítások. A Triplet MLP 30 epochon tanul standardizált bemeneten, BatchNorm és Dropout rétegekkel, hogy csökkentse a túltanulást.

6 Kiértékelési metrikák

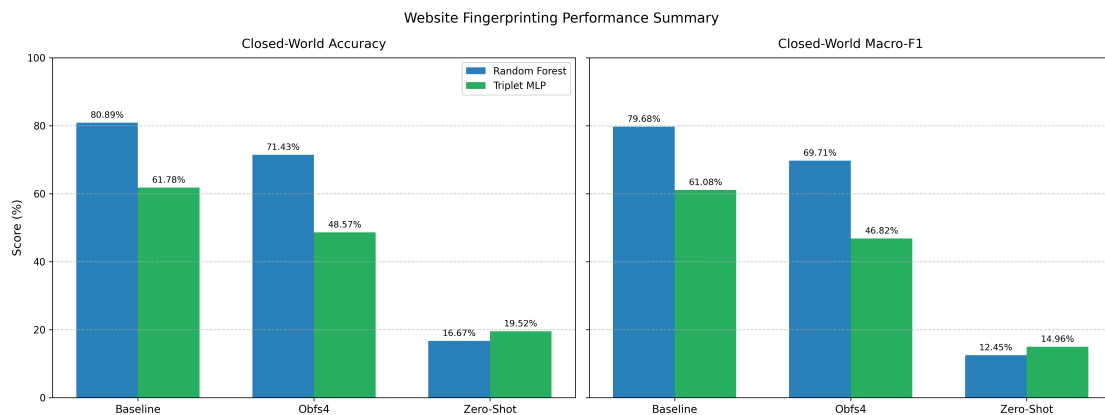
A teljesítményt pontossággal és macro-F1 értékkel mérem. A pontosság a helyes predikciók aránya, a macro-F1 pedig az osztályonként számolt F1-értékek átlaga:

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i], \quad \text{MacroF1} = \frac{1}{C} \sum_{c=1}^C \text{F1}_c. \quad (1)$$

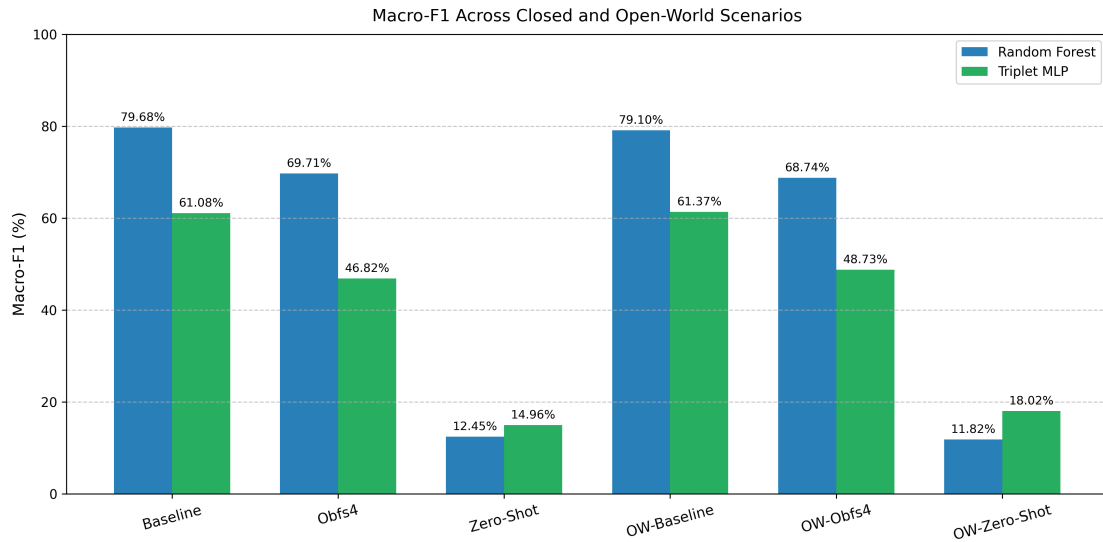
Open-world esetben külön vizsgálom az „Other” osztály recall értékét is, mivel ez mutatja, hogy az ismeretlen forgalmat mennyire képes a modell leválasztani.

7 Ábrák és vizuális összegzés

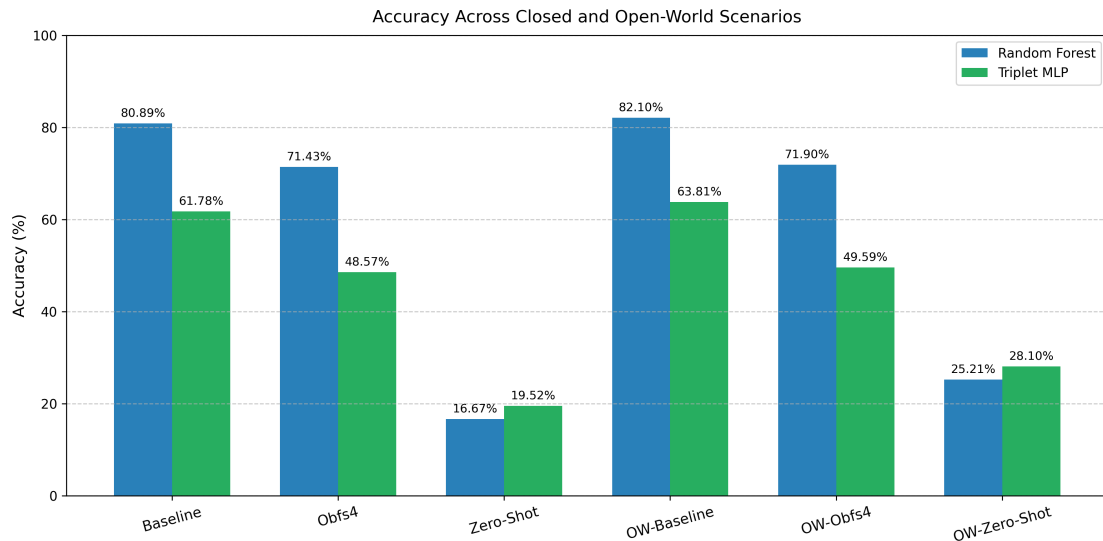
A `generate_all_figures.py` szkript a mentett metrikákból generálja a végső ábrákat, ezért az ábrák és a táblázatok ugyanabból a kiértékelési kimenetből készülnek. Az ábrák célja, hogy a fő trendeket gyorsan összegezze, majd részletesebb bontást adjon forgatókönyv, feature track és osztályszintű viselkedés szerint.



1. ábra. Zárt világú teljesítmény-összegzés (pontosság és macro-F1) a két modellre.



2. ábra. Macro-F1 alakulása minden forgatókönyvben, zárt és nyílt világú esetekben.



3. ábra. Pontosság alakulása minden forgatókönyvben, zárt és nyílt világú esetekben.

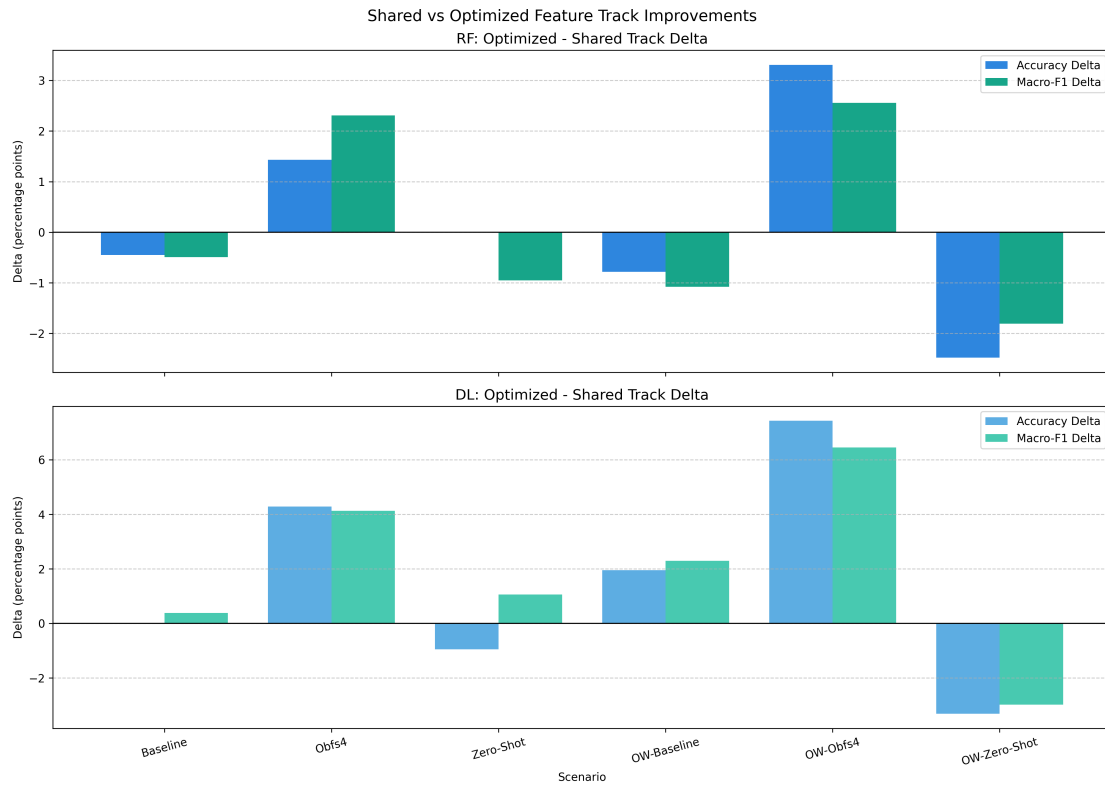
8 Kísérleti beállítások

A tanítás és kiértékelés rétegzett tanító/teszt felosztással történik. A zárt világú osztályoknál 80%/20% arányt használok, míg az open-world „Other” osztályt 50%–50% arányban bontom az ismeretlen forgalomra való általánosítás hangsúlyozására. A stabilitást három véletlen mag biztosítja (42, 52, 62). Az eredményeket átlag $\pm$ szórás formában közlöm. A metrikák a pontosság (accuracy) és a macro-F1. A zero-shot beállításban a baseline adatokon tanított modelleket obfs4 teszt adatokon értékelem. A táblázatok a *shared* track eredményeit mutatják.

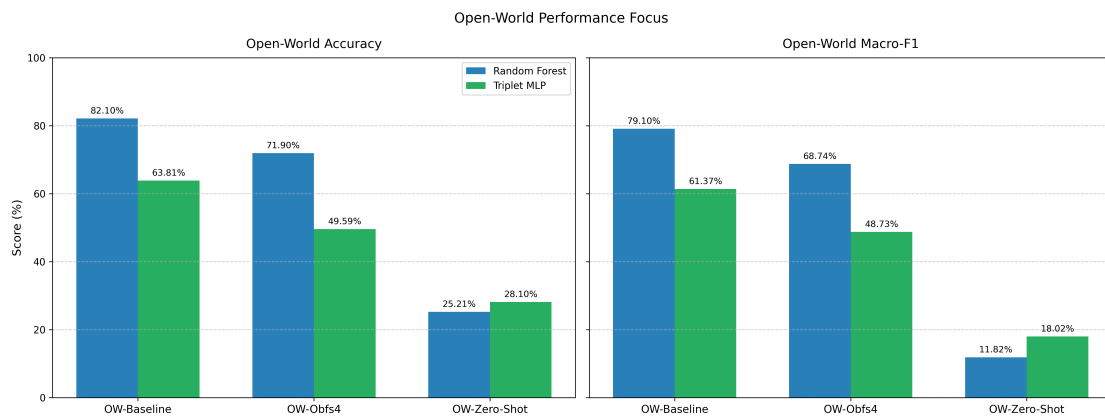
9 Reprodukálhatóság és futtatás

A pipeline fő lépései az alábbi parancsokkal futtathatók:

- `python src/extract_all_features.py`



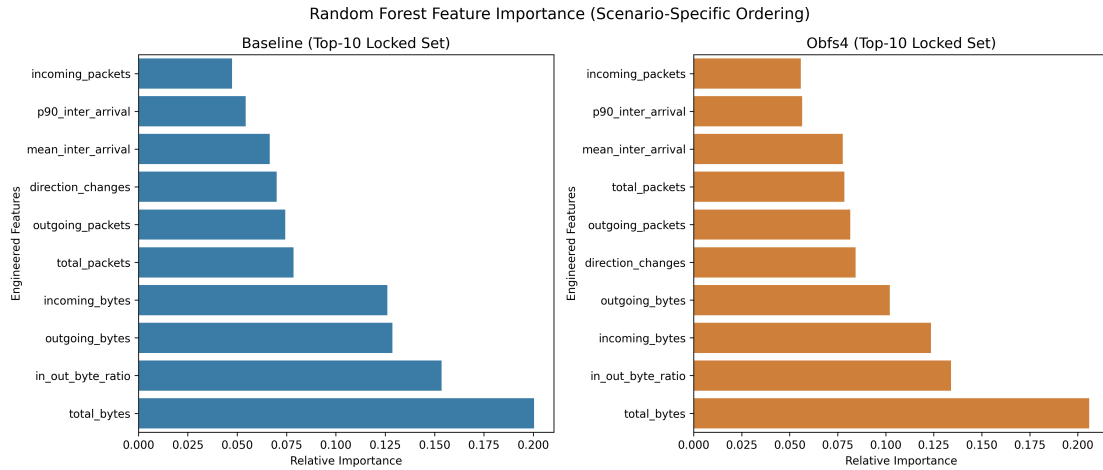
4. ábra. A *shared* és *optimized* track-ek közti különbségek (delta) pontosságban és macro-F1-ben.



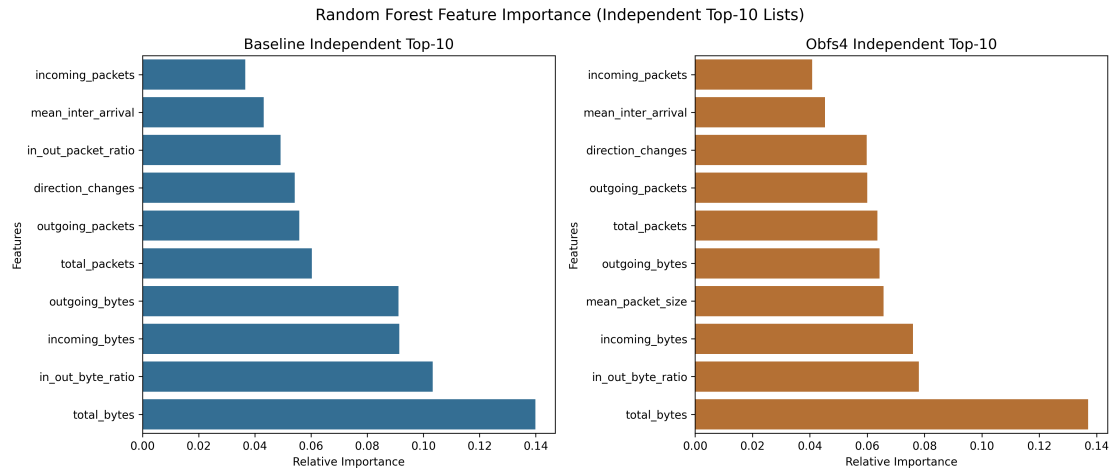
5. ábra. Nyílt világú forgatókönyvek összehasonlítása.

- `python src/evaluate_all_rf.py`
- `python src/evaluate_all_dl.py`
- `python src/generate_all_figures.py`

10 Eredmények



6. ábra. A top-10 jellemzők fontossága a közös (shared) kiválasztás mellett.



7. ábra. Forgatókönyvenként optimalizált top-10 jellemzők összehasonlítása.

2. táblázat. Zárt világú (Standard) forgatókönyvek eredményei.

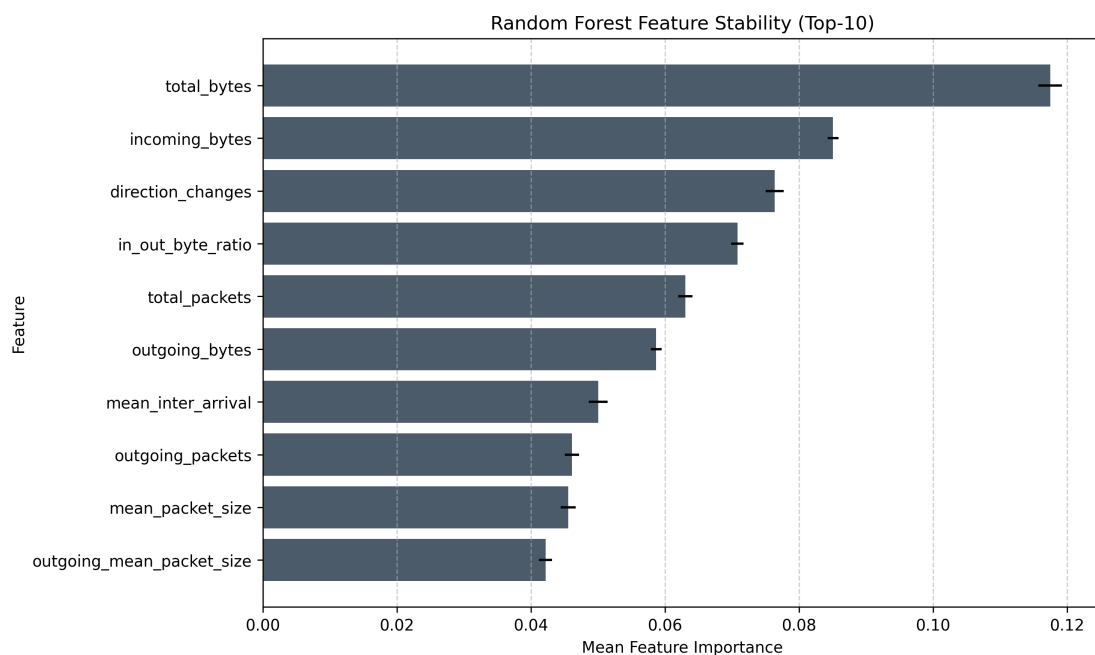
Forgatókönyv	RF Acc.	RF Macro-F1	DL Acc.	DL Macro-F1
Baseline	80.44±0.96	79.76±0.99	61.19±2.22	60.23±2.47
Obfs4	73.33±1.78	71.88±1.57	50.64±2.59	49.48±2.70
Zero-Shot	15.08±1.37	10.83±1.57	23.01±2.59	18.54±2.64

3. táblázat. Nyílt világú (Open-World) forgatókönyvek eredményei.

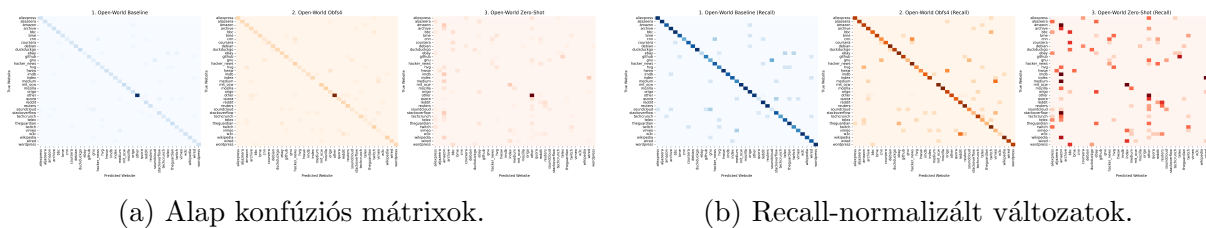
Forgatókönyv	RF Acc.	RF Macro-F1	DL Acc.	DL Macro-F1
OW-Baseline	80.93±0.84	79.11±0.68	62.00±1.32	60.37±0.76
OW-Obfs4	74.10±1.59	71.03±1.65	48.90±2.58	48.46±1.78
OW-Zero-Shot	22.73±1.79	10.41±1.00	27.27±1.79	18.42±0.41

11 Elemzés és megvitatás

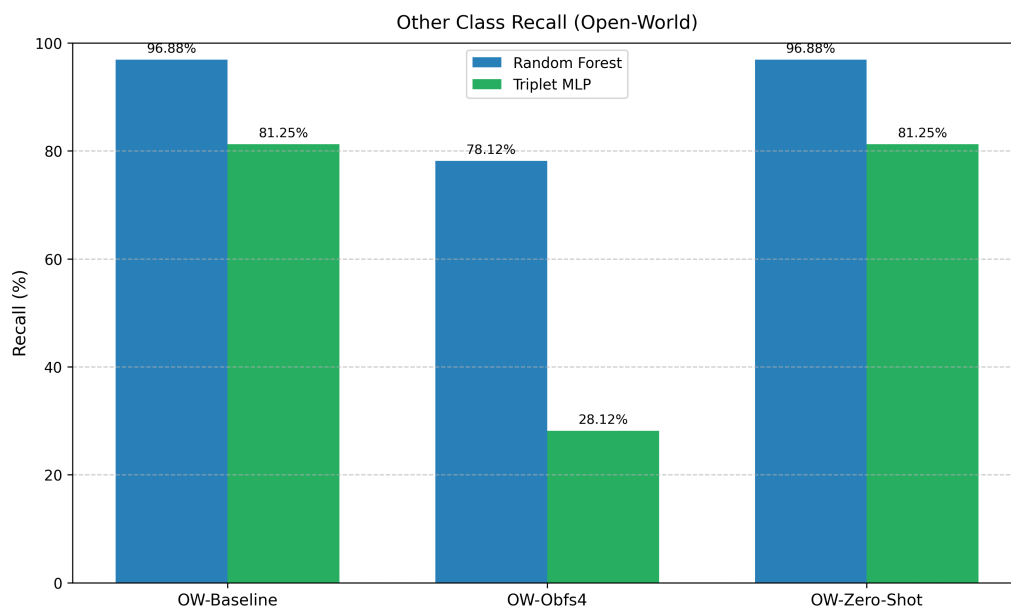
A Random Forest zárt világú baseline és obfs4 környezetben egyértelműen magasabb pontosságot és macro-F1-t ér el, ami a kézzel tervezett aggregált jellemzők hatékony ki-



8. ábra. A top-10 jellemzők stabilitása Random Forest esetén (átlag $\pm$ szórás a több futásból).



9. ábra. Konfúziós mátrixok a főbb forgatókönyvekben (shared track, seed=42).



10. ábra. Az „Other” osztály recall értékei az open-world forgatókönyvekben.

használatát jelzi (lásd 1 és 3). A zero-shot beállításban mindkét modell teljesítménye drasztikusan romlik, különösen open-world esetben, ami a domain shift egyik legfontosabb gyakorlati kihívását mutatja (lásd 2 és 5). A Triplet MLP ugyanakkor stabilabb zero-shot értékeket ad (Standard Zero-Shot: DL 23.01% vs. RF 15.08%), ami a beágyazások általánosíthatóságára utal. Az *optimized* track-ek több forgatókönyvben javítást hoznak, de a javulás nem egyenletes, ezért a *shared* beállítás jó kompromisszumot ad a konszisztencia és teljesítmény között (lásd 4). A stabil top-10 lista azt sugallja, hogy a leginformatívabb aggregált jellemzők robusztusak a tanító/teszt felosztás változására (lásd 8). Az open-world forgatókönyvekben a recall-normalizált konfúziós mátrixok jól kiemelik, hogy az „Other” osztályt a modellek gyakran sikeresen külön választják a zárt világú osztályoktól, ugyanakkor ez nem feltétlenül jelent magas precíziót (lásd 9b és 10).

12 Korlátok

- A vizsgálat aggregált folyamjellemzőkre épül, ami részben elfedheti a finom időbeli mintázatokat.
- Az open-world „Other” osztály sokféle weboldal forgalmát fogja össze, ezért a modell nehezebben választja el a zárt világú osztályoktól, ami csökkentheti a precíziós mutatókat.
- A top-10 jellemzők statikus kiválasztása miatt a ritkább, de informatív mintázatok elveszhetnek.

13 Következtetések és jövőbeli munka

A projekt igazolja, hogy a klasszikus és mélytanulási modellek eltérő erősségekkel rendelkeznek a Tor WF feladatban. A Random Forest magas baseline teljesítményt ad, míg a Triplet MLP zero-shot helyzetekben jobb általánosítást mutat. A jövőben a drift-robusztus tanítás (az időben változó forgalmi eloszláshoz igazított tréning, pl. időablakos validálás vagy domain-invariáns reprezentációk) és az adaptív újratanulás (drift detektálás után periodikus vagy online frissítés friss adatokból) lehet indokolt. Emellett a finomabb idősoros jellemzők integrálása is ígéretes (csomag-idősor, burst-mintázatok, n-gram vagy embedding alapú leírás), mert ezek érzékenyebben ragadják meg a forgalom dinamikáját. Végül érdemes a modellfrissítés költség-hatékonyságát is vizsgálni: mennyi új adat és számítási idő szükséges adott teljesítmény-nyereséghez, és milyen frissítési gyakoriság adja a legjobb kompromisszumot.

Hivatkozások

- [1] Cherubin, G., et al. (2022). *A study on concept drift in website fingerprinting*.
- [2] Sirinam, P., et al. (2019). *Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning*.